

Unit -6 : Listview, Gridview and Recyclerview [6 HRS]

Listview (Introduction, Features, Implementing listview in an application)

ListView is a view in Android that displays a vertical, scrollable list of items. Each item in the list is managed and populated by an Adapter. The Adapter retrieves data from a source such as an array or a database and provides it to the ListView for display.

Features:

- **Single Column Layout:**
 - **Description:** `ListView` displays items in a single vertical column. This layout is ideal for lists where each item is shown sequentially in a vertical format.
 - **Example:** In a shopping app, you might have a list of products where each product is shown in a vertical list with its name, price, and image. As users scroll down, more products are loaded in the same vertical column.
- **Adapter-Based:**
 - **Description:** `ListView` utilizes an `Adapter` to bridge the data source (e.g., an array or database) with the UI. The `Adapter` fetches data from the source and provides it to the `ListView` for display.
 - **Example:** In a news app, an `Adapter` retrieves news article details such as the title, description, and image from a database and populates each row of the `ListView` with this data.
- **Item Click Handling:**
 - **Description:** `ListView` supports handling click and long-click events on individual list items. This feature allows interactions such as item selection or performing actions when an item is clicked.
 - **Example:** In a contact management app, each contact in the list can be clicked to view detailed information about that contact. Long-clicking a contact might provide options to delete or edit the contact.
- **Header and Footer Views:**
 - **Description:** `ListView` allows adding header and footer views to the list. This is useful for including additional content or controls above or below the list of items.
 - **Example:** In a social media app, a `ListView` showing a user's posts can have a header view with the user's profile picture and name, and a footer view with a button to load more posts or add a new post.
- **Efficient Data Handling:**
 - **Description:** `ListView` efficiently manages memory for large datasets by recycling views. As items scroll out of view, their views are reused for new items coming into view, which helps in handling large data sets smoothly.
 - **Example:** In a weather app, if you have a list showing daily weather forecasts for a month, `ListView` will recycle views as you scroll through the list, ensuring smooth scrolling and efficient memory use.
- **Vertical Scrolling:**
 - **Description:** `ListView` provides default vertical scrolling, making it suitable for displaying long lists of items where users need to scroll to view all content.

- **Example:** In a movie streaming app, `ListView` can display a list of recommended movies, allowing users to scroll vertically to see the entire list of recommendations without additional scroll bars or controls.

Implementing ListView in an Application:

A step-by-step guide to implementing a `ListView` in an Android application using the default layout:

1. Define the ListView in XML Layout:

In your XML layout file (e.g., `activity_main.xml`), define the `ListView` with the desired width and height. Here's an example:

```
<ListView
    android:id="@+id/listView"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

2. Create and Set an Adapter:

In your activity class (e.g., `MainActivity.java`), initialize the `ListView` and create an `ArrayAdapter` to bind data to the `ListView`. Here's how you can do it:

```
// Import statements
import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import android.widget.Toast;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Find the ListView by its ID
        ListView listView = findViewById(R.id.listView);

        // Define an array of items
        String[] food_listItem = {"Fulki", "Pani Puri", "Chawmin",
        "PIZZA", "MOMO", "Samosa", "Pakauda"};

        // Create and set an ArrayAdapter to the ListView
        ArrayAdapter<String> adapter = new ArrayAdapter<>(
            this, // Context
            android.R.layout.simple_list_item_1, // Layout for
individual items
            food_listItem // Data source
        );
```

```

        listView.setAdapter(adapter);
    }
}

```

3. Handle Item Clicks:

To handle click events on list items, set an `OnItemClickListener` for the `ListView`. This allows you to perform actions when a user clicks on an item. Here's an example:

```

listView.setOnItemClickListener(new AdapterView.OnItemClickListener() {
    @Override
    public void onItemClick(AdapterView<?> parent, View view, int
position, long id) {
        // Get the clicked item
        String item = (String) parent.getItemAtPosition(position);

        // Show a Toast message with the clicked item
        Toast.makeText(MainActivity.this, "Clicked: " + item,
Toast.LENGTH_SHORT).show();
    }
});

```

Complete Implementation in Context

Putting it all together, your `MainActivity.java` file should look something like this:

```

import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Find the ListView by its ID
        ListView listView = findViewById(R.id.listView);

        // Define an array of items
        String[] food_listItem = {"Fulki", "Pani Puri", "Chawmin",
"PIZZA", "MOMO", "Samosa", "Pakauda"};

        // Create and set an ArrayAdapter to the ListView
        ArrayAdapter<String> adapter = new ArrayAdapter<>(
            this, // Context
            android.R.layout.simple_list_item_1, // Layout for
individual items

```

```

        food_listItem // Data source
    );

    listView.setAdapter(adapter);

    // Handle item clicks
    listView.setOnItemClickListener(new
    AdapterView.OnItemClickListener() {
        @Override
        public void onItemClick(AdapterView<?> parent, View view,
        int position, long id) {
            // Get the clicked item
            String item = (String)
            parent.getItemAtPosition(position);

            // Show a Toast message with the clicked item
            Toast.makeText(MainActivity.this, "Clicked: " + item,
            Toast.LENGTH_SHORT).show();
        }
    });
}
}
}

```

Gridview (Introduction, Features, Implementing gridview in an application)

GridView is a view in Android that displays items in a two-dimensional grid layout, consisting of rows and columns. It is a flexible and efficient way to present data in a grid format where the items are dynamically inserted and managed.

Features:

- **Two-Dimensional Layout:**
 - **Description:** **GridView** displays items in a grid format with both rows and columns. This allows for a structured layout where items are organized into a matrix-like view.
 - **Example:** An image gallery app uses **GridView** to display photos in a grid, allowing users to view and interact with multiple images in a structured layout.
- **Adapter-Based:**
 - **Description:** **GridView** uses an **Adapter** to bind data to the grid. The **Adapter** serves as a bridge between the data source (such as an array or database) and the **GridView**, handling the conversion of data into visual elements.
 - **Example:** In a contact management app, an **Adapter** might be used to populate a grid with contact profile pictures and names, retrieving data from a database and presenting it in the grid format.
- **Item Selection:**
 - **Description:** Users can select items within the **GridView** by clicking on them. This feature supports user interaction, such as selecting an item for further action or display.
 - **Example:** In a shopping app, a user might click on a product image in the grid to view more details about that product or to add it to their cart.
- **Default Scrolling:**

- **Description:** `GridView` provides vertical scrolling by default, eliminating the need for additional scroll views. This allows users to scroll through the grid to see more items beyond the visible area.
- **Example:** In a video streaming app, `GridView` could display a grid of video thumbnails, and users can scroll vertically to explore more videos without needing extra scroll bars.
- **Common Use Cases:**
 - **Description:** `GridView` is widely used in applications where content is best presented in a grid layout. This includes scenarios like image galleries, product listings, and any other content that benefits from a grid arrangement.
 - **Example:** A social media app might use `GridView` to show a user's photo albums in a grid, making it easy to browse through multiple images or posts in a structured format.

Implementing `GridView` in an Application

Here's a step-by-step guide to implementing a `GridView` in an Android application:

1. Define the `GridView` in XML Layout

In your XML layout file (e.g., `activity_main.xml`), define the `GridView` with the desired width and height. Here's an example:

```
<GridView
    android:id="@+id/gridView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:numColumns="auto_fit"
    android:columnWidth="100dp"
    android:verticalSpacing="10dp"
    android:horizontalSpacing="10dp"
    android:stretchMode="columnWidth"
    android:gravity="center"/>
```

- **`android:numColumns`:** Defines the number of columns in the grid. `auto_fit` will automatically adjust the number of columns based on the available space.
- **`android:columnWidth`:** Sets the width of each column in the grid.
- **`android:verticalSpacing` and `android:horizontalSpacing`:** Define the space between items.
- **`android:stretchMode`:** Defines how the grid items are stretched.

2. Create and Set an Adapter

In your activity class (e.g., `MainActivity.java`), initialize the `GridView` and create an `ArrayAdapter` to bind data to the `GridView`. Here's how you can do it:

```
// Import statements
import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
```

```

import android.widget.AdapterView;
import android.widget.GridView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Find the GridView by its ID
        GridView gridView = findViewById(R.id.gridView);

        // Define an array of items (e.g., image resource IDs or strings)
        String[] items = {"Apple", "Banana", "Cherry", "Date", "Fig", "Grape", "Kiwi"};

        // Create and set an ArrayAdapter to the GridView
        ArrayAdapter<String> adapter = new ArrayAdapter<>(
            this,                // Context
            android.R.layout.simple_list_item_1, // Layout for individual items
            items                // Data source
        );

        gridView.setAdapter(adapter);
    }
}

```

3. Handle Item Clicks

To handle click events on grid items, set an `OnItemClickListener` for the `GridView`. This allows you to perform actions when a user clicks on an item. Here's an example:

```

gridView.setOnItemClickListener(new AdapterView.OnItemClickListener() {
    @Override
    public void onItemClick(AdapterView<?> parent, View view, int position,
long id) {
        // Get the clicked item
        String item = (String) parent.getItemAtPosition(position);

        // Show a Toast message with the clicked item
        Toast.makeText(MainActivity.this, "Clicked: " + item, Toast.LENGTH_SHORT).show();
    }
});

```

Complete Implementation in Context

Putting it all together, your `MainActivity.java` file should look like this:

```
import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.AdapterView.OnItemClickListener;
import android.widget.ArrayAdapter;
import android.widget.GridView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Find the GridView by its ID
        GridView gridView = findViewById(R.id.gridView);

        // Define an array of items (e.g., image resource IDs or strings)
        String[] items = {"Apple", "Banana", "Cherry", "Date", "Fig", "Grape", "Kiwi"};

        // Create and set an ArrayAdapter to the GridView
        ArrayAdapter<String> adapter = new ArrayAdapter<>(
            this, // Context
            android.R.layout.simple_list_item_1, // Layout for individual items
            items // Data source
        );

        gridView.setAdapter(adapter);

        // Handle item clicks
        gridView.setOnItemClickListener(new AdapterView.OnItemClickListener() {
            @Override
            public void onItemClick(AdapterView<?> parent, View view, int
position, long id) {
                // Get the clicked item
                String item = (String) parent.getItemAtPosition(position);

                // Show a Toast message with the clicked item
                Toast.makeText(MainActivity.this, "Clicked: " + item,
                    Toast.LENGTH_SHORT).show();
            }
        });
    }
}
```

```
}  
}
```

Recyclerview (Introduction, Features, Implementing recyclerview in an application)

`RecyclerView` is a more advanced and flexible version of `ListView` and `GridView`, designed for displaying large sets of data efficiently in Android. It is highly customizable and can handle different types of layouts, including linear lists, grids, and staggered grids.

Features:

- **Flexible Layouts:**
 - **Description:** `RecyclerView` supports various layout managers, including linear, grid, and staggered grid layouts. This flexibility allows you to display data in different formats according to the needs of your application.
 - **Example:** In a news app, you could use a `LinearLayoutManager` to display a vertical list of news articles or a `GridLayoutManager` to show articles in a grid format.
- **Adapter-Based:**
 - **Description:** Similar to `ListView` and `GridView`, `RecyclerView` uses an `Adapter` to bind data to the view. The `Adapter` provides the data and creates view holders to represent each item in the dataset.
 - **Example:** In a movie app, an `Adapter` retrieves movie details (like title, poster, and rating) from a database and binds this data to the `RecyclerView` using custom view holders.
- **ViewHolder Pattern:**
 - **Description:** `RecyclerView` utilizes the `ViewHolder` pattern to improve performance by recycling view items and reducing the number of calls to `findViewById()`. This pattern helps manage large datasets more efficiently.
 - **Example:** In a chat application, each message in a conversation is represented by a view holder. As messages scroll off-screen, their view holders are recycled for new messages, ensuring smooth scrolling and memory efficiency.
- **Item Decoration and Animation:**
 - **Description:** `RecyclerView` allows for custom item decorations and animations. You can add dividers, spacers, or other decorations between items and animate item changes.
 - **Example:** In a contact list app, you might use item decorations to add separators between contacts and animations to smoothly transition when contacts are added or removed.
- **Item Click Handling:**
 - **Description:** `RecyclerView` supports item click events through its `Adapter` and `ViewHolder`. This enables interaction with individual items in the list or grid.
 - **Example:** In a gallery app, users can click on images in the `RecyclerView` to view a larger version of the image or perform actions like sharing or editing.

- **Default Scrolling:**

- **Description:** RecyclerView provides efficient vertical and horizontal scrolling by default. It is optimized for handling large datasets with smooth scrolling.
- **Example:** In an e-commerce app, RecyclerView can display a long list of product categories or a horizontal list of featured products with smooth scrolling.

1. Implementing RecyclerView in an Application

In your XML layout file (e.g., `activity_main.xml`), define the RecyclerView with the desired width and height:

```
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
```

2. Create Item Layout

Define the layout for each item in the RecyclerView (e.g., `res/layout/item_food.xml`):

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:id="@+id/textViewFoodName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="18sp"
        android:textColor="#000" />

    <TextView
        android:id="@+id/textViewFoodPrice"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="18sp"
        android:text="200"
        android:textColor="#000" />

</LinearLayout>
```

3. Implement RecyclerView in Fragment

Create a Fragment to set up and use the RecyclerView:

```

package com.example.myapplication;

import android.os.Bundle;
import androidx.fragment.app.Fragment;
import androidx.recyclerview.widget.LinearLayoutManager;
package com.example.myapplication;

import android.os.Bundle;
import androidx.fragment.app.Fragment;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.GridLayoutManager;
import androidx.recyclerview.widget.RecyclerView;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

public class RecyclerViewFragment extends Fragment {

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup
container,
                                Bundle savedInstanceState) {
        // Inflate the layout for this fragment
        View view = inflater.inflate(R.layout.fragment_recycler_view,
container, false);

        String[] food_listItem = {"Fulki", "Pani Puri", "Chawmin",
"PIZZA", "MOMO", "Samosa", "Pakauda"};
        String[] food_listItemPrice = {"200", "300", "400", "500",
"600", "700", "800"};

        RecyclerView recyclerView =
view.findViewById(R.id.recyclerview_example);
        recyclerView.setLayoutManager(new
LinearLayoutManager(getActivity())); // Set the layout manager
        // Uncomment the following line to use GridLayoutManager
        // recyclerView.setLayoutManager(new
GridLayoutManager(getActivity(), 2));

        // Set the adapter
        recyclerView.setAdapter(new FoodAdapter(food_listItem,
food_listItemPrice));

        return view;
    }

    // Adapter class
    private static class FoodAdapter extends
RecyclerView.Adapter<FoodAdapter.FoodViewHolder> {

        private final String[] foodList;
        private final String[] foodPriceList;

```

```

        FoodAdapter(String[] foodList, String[] foodPriceList) {
            this.foodList = foodList;
            this.foodPriceList = foodPriceList;
        }

        @NonNull
        @Override
        public FoodViewHolder onCreateViewHolder(@NonNull ViewGroup
parent, int viewType) {
            View itemView = LayoutInflater.from(parent.getContext())
                .inflate(R.layout.item_food, parent, false);
            return new FoodViewHolder(itemView);
        }

        @Override
        public void onBindViewHolder(@NonNull FoodViewHolder holder,
int position) {
            holder.foodNameTextView.setText(foodList[position]);
            holder.foodPriceTextView.setText(foodPriceList[position]);
        }

        @Override
        public int getItemCount() {
            return foodList.length;
        }

        static class FoodViewHolder extends RecyclerView.ViewHolder {
            TextView foodNameTextView;
            TextView foodPriceTextView;

            FoodViewHolder(@NonNull View itemView) {
                super(itemView);
                foodNameTextView =
itemView.findViewById(R.id.textviewFoodName);
                foodPriceTextView =
itemView.findViewById(R.id.textviewFoodPrice);
            }
        }
    }
}

```

Or, Implement RecyclerView in Activity

In your Activity (e.g., MainActivity.java), set up the RecyclerView:

```

package com.example.myapplication;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.GridLayoutManager;
import androidx.recyclerview.widget.RecyclerView;
import android.view.LayoutInflater;

```

```

import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Data to display
        String[] foodList = {"Fulki", "Pani Puri", "Chawmin", "PIZZA",
"MOMO", "Samosa", "Pakauda"};
        String[] foodPriceList = {"200", "300", "400", "500", "600",
"700", "800"};

        // Set up RecyclerView
        RecyclerView recyclerView = findViewById(R.id.recyclerView);
        recyclerView.setLayoutManager(new LinearLayoutManager(this));
// Linear layout manager

        // Uncomment the following line to use GridLayoutManager
        // recyclerView.setLayoutManager(new GridLayoutManager(this,
2)); // 2 columns for grid layout

        // Set the adapter
        recyclerView.setAdapter(new FoodAdapter(foodList,
foodPriceList));
    }

    // Adapter class
    private static class FoodAdapter extends
RecyclerView.Adapter<FoodAdapter.FoodViewHolder> {

        private final String[] foodList;
        private final String[] foodPriceList;

        FoodAdapter(String[] foodList, String[] foodPriceList) {
            this.foodList = foodList;
            this.foodPriceList = foodPriceList;
        }

        @NonNull
        @Override
        public FoodViewHolder onCreateViewHolder(@NonNull ViewGroup
parent, int viewType) {
            View itemView = LayoutInflater.from(parent.getContext())
                .inflate(R.layout.item_food, parent, false);
            return new FoodViewHolder(itemView);
        }

        @Override

```

```
        public void onBindViewHolder(@NonNull FoodViewHolder holder,
int position) {
            holder.foodNameTextView.setText(foodList[position]);
            holder.foodPriceTextView.setText(foodPriceList[position]);
        }

        @Override
        public int getItemCount() {
            return foodList.length;
        }

        static class FoodViewHolder extends RecyclerView.ViewHolder {
            TextView foodNameTextView;
            TextView foodPriceTextView;

            FoodViewHolder(@NonNull View itemView) {
                super(itemView);
                foodNameTextView =
itemView.findViewById(R.id.textviewFoodName);
                foodPriceTextView =
itemView.findViewById(R.id.textviewFoodPrice);
            }
        }
    }
}
```